

1. SGLang 的 RadixCache 解决的问题是通过极其精细的内存管理（用多少给多少），加上极其聪明的复用机制（相同的前缀绝不存两遍），把大模型推理中最吃显存的 **KV Cache** 压榨到了极致。大大减少了使用的时间，简单来说，就是 SGLang 的 RadixAttention 会利用 RadixCache 查找之前缓存过的相同前缀，并复用对应的 KV。这样，模型可以跳过一部分重复计算，降低请求的处理时间。RadixCache 会管理这些缓存；显存紧张时，缓存内容也可能被释放。（见论文）
2. page-sized KV cache 对应的是流程里面“计算输入内容并生成 KV 内容”这一部分，将 KV 切割成一块一块，系统按需分配回收空间，prefix reuse 对应的是“检查之前有没有相同的开头并复用 KV”这一块。
3. vLLM 的 PagedAttention 解决的是传统 KV 缓存先预分配而导致大块连续空间造成的浪费的问题，也就是说 vLLM 的 PagedAttention 把每个请求的 KV 缓存分成固定大小的块来管理，并通过块的对应关系找到模型需要读取的内容。这样不必要求每个请求的缓存都放在一整段连续空间里，有助于更灵活地利用显存。**SGLang 的 RadixAttention / RadixCache** 重点是如何查找相同前缀，并复用之前产生的 KV。

联系：两者都围绕同一份 KV 缓存工作，目的都是让模型推理更省时间、更有效地使用显存。差异：一个重点在 KV 怎样分块和存取，一个重点在利用前缀复用。